

SE446: Big Data Engineering

Week 7A: Apache Spark — Fundamentals & RDDs

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary

Learning Objectives

By the end of this session you will be able to:

- 1 Explain why Spark outperforms MapReduce for iterative workloads
- 2 Describe the Spark cluster architecture (Driver, Master, Workers, Executors)
- 3 Distinguish **transformations** (lazy) from **actions** (eager)
- 4 Create and manipulate RDDs using PySpark
- 5 Read the Spark Web UI job execution report

Dataset

Chicago Crimes dataset on HDFS — same data we used for Hive queries.

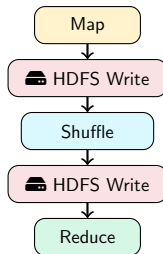
Recap: MapReduce Revisited

What MapReduce gave us:

- ✓ Horizontal scalability
- ✓ Fault tolerance
- ✓ Data locality

What MapReduce **costs**:

- ✗ Every stage writes to HDFS
- ✗ Serialize $\rightarrow$ compress $\rightarrow$ replicate $\times 3$
- ✗ Painful for iterative algorithms



The Cost of Iteration

A 10-iteration ML algorithm on MapReduce = **20 HDFS read/write cycles.**

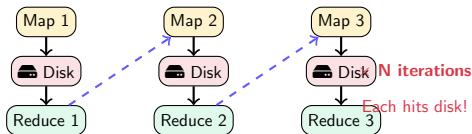
The Problem: Disk I/O Kills Iterative Algorithms

Good for MapReduce:

- One-pass ETL jobs
- Batch aggregations
- Log processing

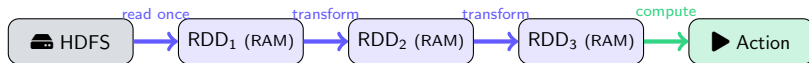
Painful for MapReduce:

- Machine learning (iterative)
- Graph algorithms (PageRank)
- Interactive ad-hoc queries



"Many algorithms require passing over the same data dozens of times. MapReduce makes this prohibitively slow." — Zaharia et al., 2010

The Solution: Keep Data in Memory



Core idea: Instead of writing intermediate results to disk, keep them in **RAM**.

RDD = Resilient Distributed Dataset

- Read-only, partitioned, in-memory collection
- Fault tolerance by **lineage** (Spark remembers the sequence of transformations and recomputes lost partitions)
- No need to replicate data across nodes

Key Statistic

Spark is **10–100×** faster than MapReduce on iterative workloads

What Is an RDD? — Think of It Like a Recipe

Simple Analogy:

Your RDD

- Read data from HDFS
- Filter out bad rows
- Group by key
- Sum values

→ **Get result**

(not executed yet!)

This is your **RDD** — a distributed dataset defined by a *recipe* of transformations.

Three Key Properties:

1 Distributed

Data is split into chunks (partitions), one per task. If your file is 400 MB and you have 4 workers, each gets ≈ 100 MB.

2 Resilient

If Worker 1 crashes, Spark *stores* the transformation history (lineage) and recomputes it from scratch.

3 Lazy

Nothing happens when you write `rdd = data.map(...)`

Spark only executes when you call an *action* (`.count()`, `.collect()`)

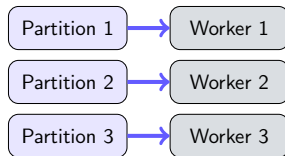
Remember

Transformations are lazy recipes; actions cook the meal. RDD = a snapshot of your data + the instructions to rebuild it if needed.

How Spark RDD Actually Works

1. Distributed Processing

- Data is split into **partitions**
- Each worker (executor) processes its own partition
- Data is loaded **in memory per partition**
- Parallel execution across multiple machines



If Worker 2 crashes → recompute Partition 2

2. Fault Tolerance via Lineage

- Spark records the **sequence of transformations**
- If a worker crashes, only the **lost partition** is recomputed
- No need to replicate full datasets

3. Lazy Evaluation

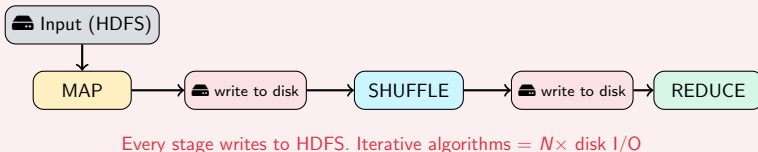
- Transformations build a **logical plan (DAG)**
- Execution starts only when an **action** is called

RDD in One Sentence

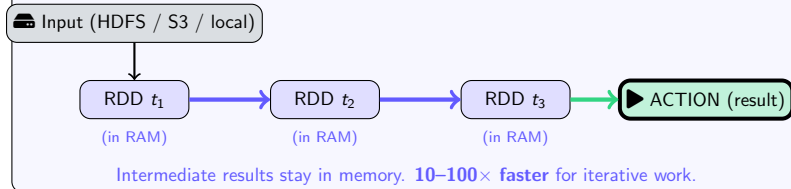
A distributed, immutable dataset processed in parallel, lazily evaluated, and fault tolerant via lineage.

MapReduce vs Spark — Side by Side

MapReduce (Disk-Based)



Spark (In-Memory DAG)

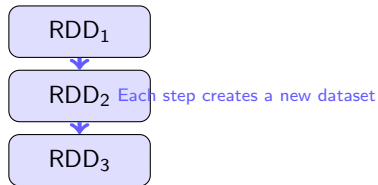


Is Each RDD a Dataset?

Yes — each RDD is a distributed dataset.

Example:

- `rdd1 = sc.textFile("data.txt")`
- `rdd2 = rdd1.filter(...)`
- `rdd3 = rdd2.map(...)`



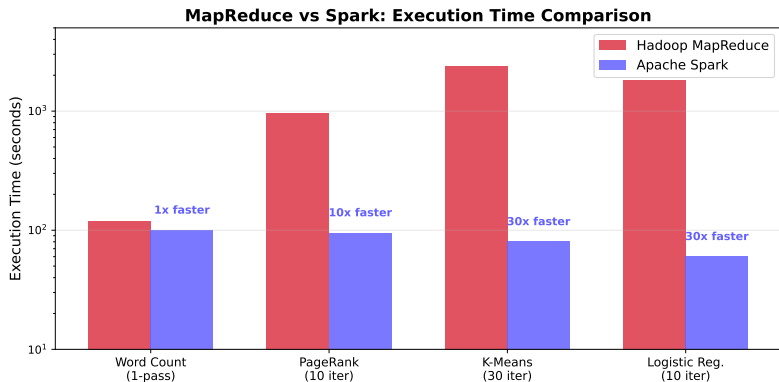
What this means:

- Each RDD is a **new dataset**
- RDDs are **immutable**
- Data is **partitioned across nodes**
- Defined by a sequence of transformations (lineage)

Important

An RDD is not a file. It is a logical, distributed dataset that may be materialized in memory after an action.

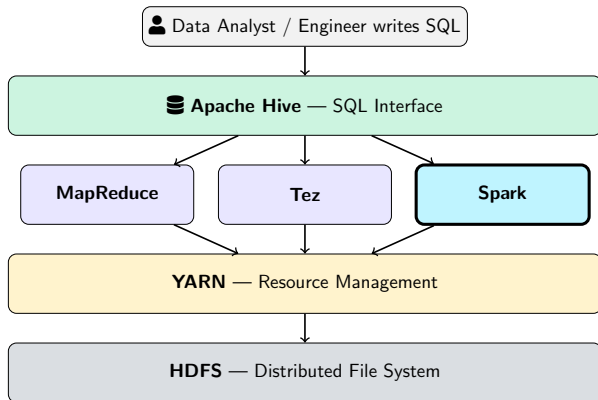
MapReduce vs. Spark: Performance Comparison



Why the Difference?

Spark keeps intermediate data in **memory**. Iterative algorithms (ML, PageRank) re-read the same data — Spark avoids disk I/O on every pass.

Where Spark Fits in the Hadoop Ecosystem



Important Nuance

This diagram shows one common deployment where Spark runs **inside the Hadoop stack**. But Spark itself is **not** tied to Hadoop — as the next slides explain.

Clarification: Spark Is an Independent Project

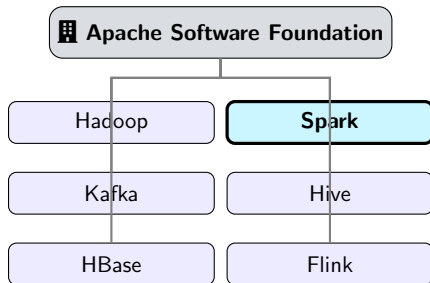
Common Misconception

Students often assume Spark **requires** Hadoop, HDFS, or YARN.

It does not.

Spark is an **independent** project under the **Apache Software Foundation** — the same foundation that hosts Hadoop, Kafka, Hive, and HBase.

These are **sibling projects**, not parent–child:

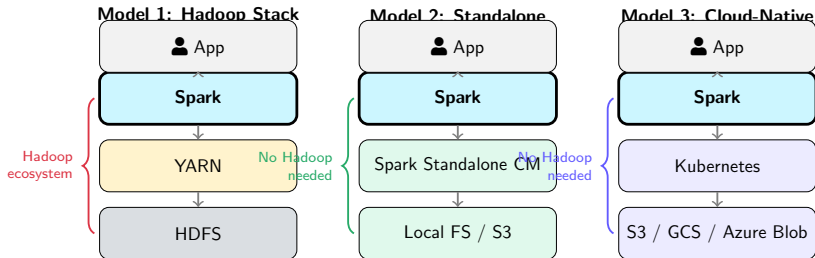


Analogy

Think of the Apache Foundation as a **university**. Hadoop, Spark, Kafka, and Hive are different **departments** — they can collaborate, but each runs independently.

Spark's Deployment Models

Spark only requires two things: a **cluster manager** and a **storage layer**.
Neither must be Hadoop.



Takeaway

The Hadoop stack (HDFS + YARN) is just **one** of many deployment options. Most modern production systems use **Kubernetes + S3** with no Hadoop at all.

Why the Hadoop Association Persists

🕒 Historical context:

- 1 Spark was born at UC Berkeley (2009) as an improvement over MapReduce
- 2 Early clusters already had HDFS + YARN
- 3 Spark **replaced MapReduce** in many Hadoop deployments
- 4 Textbooks & tutorials still show the Hadoop stack

⚠️ This is why students assume Spark = part of Hadoop.

🚀 Modern reality (2024+):

- Databricks runs Spark on **cloud object stores**
- AWS EMR Serverless: Spark + S3 (no HDFS)
- Google Dataproc: Spark + GCS
- Azure Synapse: Spark + Azure Data Lake
- Many companies: **Spark + Kubernetes**

✓ Hadoop is optional — Spark is standalone.

🗣️ Professor's Summary

Apache Spark is an independent distributed computing framework. While historically integrated into the Hadoop ecosystem (HDFS + YARN), it can operate **standalone**, with its own cluster manager, and integrate with **any** storage system — HDFS, S3, local files, or databases.

So Why Did We Show the Hadoop Diagram?

⚠ Context for This Course

In SE446, our lab cluster runs **HDFS + YARN + Spark**.

The ecosystem diagram reflects **our lab setup** — one valid deployment model.

What Spark needs (minimal):

- ✓ A cluster manager
(Standalone / YARN / K8s)
- ✓ A storage layer
(local FS / S3 / HDFS / DB)

That's it. Two components.

Spark does NOT require:

- ✗ HDFS
- ✗ YARN
- ✗ Hive
- ✗ MapReduce
- ✗ Any other Hadoop component

🔑 The Bottom Line

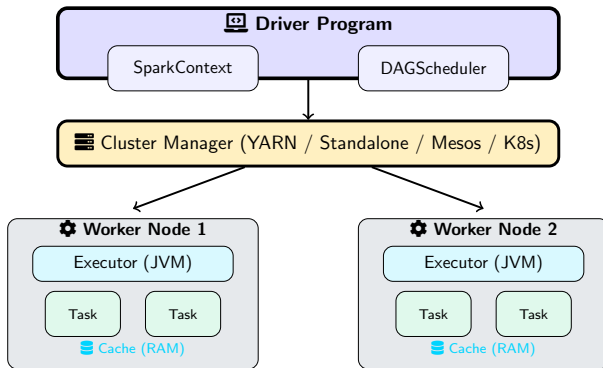
Our lab uses the Hadoop stack for convenience, but remember:

Spark is a standalone system that can plug into any infrastructure.

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary

Spark Cluster Architecture Overview



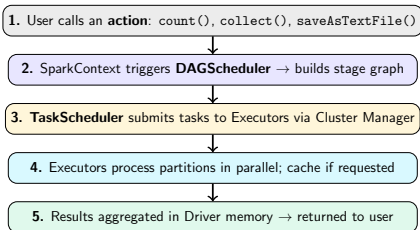
Component Roles — Deep Dive

Component	Role
 Driver	Hosts SparkContext; builds DAG; schedules stages
 SparkContext	Entry point to Spark; connects to cluster manager
 Cluster Manager	Allocates resources (YARN on our cluster)
 Executor	JVM process on each worker; runs tasks; holds cache
 Task	Smallest unit of work — runs on one partition
 Cache	In-memory storage for persisted RDDs

Key Difference from MapReduce

In MapReduce, each task runs in its own JVM (slow startup). In Spark, an **Executor JVM** stays alive for the whole application and runs **many tasks** — much lower overhead.

How a Spark Job Runs — Step by Step



Key Terms

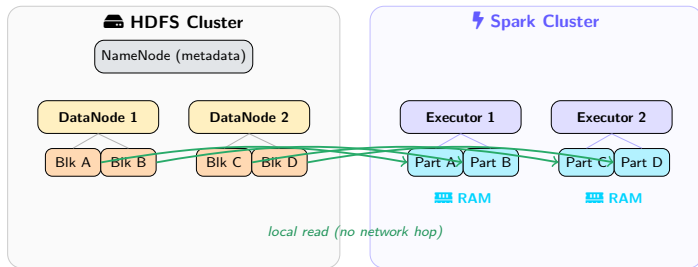
- **DAG**
Directed Acyclic Graph of transformations
- **Stage**
A set of narrow transformations without shuffle
- **DAGScheduler**
Splits DAG at shuffle boundaries
- **TaskScheduler**
Sends tasks to executors
- **Executor**
Worker process that runs tasks

Stages and Shuffles

A **stage boundary** occurs whenever data must be **shuffled** across partitions (e.g., `reduceByKey`, `groupByKey`, `join`).

Within a stage, tasks pipeline transformations without writing to disk.

How Does Spark Load Data into Memory?



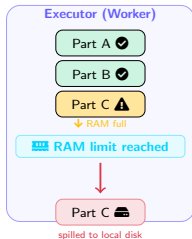
- ✓ **Data locality** — task runs on the machine that holds its block.
- ✓ **Partitioning** — 1 HDFS block $\Rightarrow$ 1 RDD partition in RAM.
- ✓ **Lazy loading** — data enters RAM only when an **action** is called.
- ✓ **Selective caching** — use `.cache()` explicitly; nothing loaded by default.

Bottom Line — No machine loads the whole dataset

Worker 1 holds Blk A & B $\rightarrow$ Executor 1 loads them into RAM. Worker 2 holds Blk C & D $\rightarrow$ Executor 2 loads them into RAM. Processing is **parallel** and **local**.

What If the Worker Runs Out of Memory?

Spark's answer: **Spill to Disk**



Choose a Storage Level when caching:

Level	Meaning
MEMORY_ONLY	Store in RAM; recompute lost partitions. Default. Fast if RAM fits.
MEMORY_AND_DISK	RAM first; spill overflow to disk. Slower reads, but no recomputation.
DISK_ONLY	Always on disk. Use only as last resort.
MEMORY_ONLY_SER	Serialized in RAM. Less space, more CPU.

How to set it in PySpark:

```
from pyspark import StorageLevel
rdd.persist(StorageLevel.MEMORY_AND_DISK)
```

Rule of Thumb

Use `MEMORY_ONLY` by default. If you see `SpillMetrics` in the Spark UI or out-of-memory errors, switch to `MEMORY_AND_DISK` or allocate more executor memory: `--executor-memory 4g`.

Spark on Our Cluster

Our environment: Hadoop 3.4.1 + YARN, 1 Master + 2 Workers

Starting PySpark (interactive shell):

```
pyspark --master yarn --deploy-mode client \  
        --num-executors 2 --executor-memory 1g
```

Submitting a script:

```
spark-submit --master yarn --deploy-mode client \  
            --num-executors 2 --executor-memory 1g \  
            my_spark_job.py
```

Web UIs

- YARN RM: <http://master-node:8088>
- Spark UI: <http://master-node:4040>
(active only while a job is running)

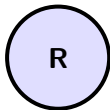
Deploy Modes

client: Driver on your machine (for dev)
cluster: Driver on a worker (for prod)

Outline

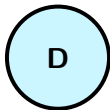
- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary

What Is an RDD?



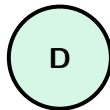
Resilient

Recomputed on failure
via **lineage**



Distributed

Partitioned across
cluster nodes



Dataset

Read-only collection
of records

Key Properties:

- **Immutable** — transformations create a new logical dataset without modifying or copying existing data.
- **Lazily evaluated** — Spark builds a DAG of transformations and executes it only when an action is called.
- **Partitioned** — the dataset is divided into partitions, each processed in parallel by a task.
- **Memory model** — data is loaded into RAM only during execution (or when explicitly cached); intermediate RDDs are not stored by default.

Creating RDDs — Three Ways

```
# 1. From a Python collection (parallelized)
rdd = sc.parallelize([1, 2, 3, 4, 5], numSlices=2)

# 2. From a local file
rdd = sc.textFile("file:///path/to/local.csv")

# 3. From HDFS (most common in production)
rdd = sc.textFile("hdfs:///data/chicago_crimes.csv")
```

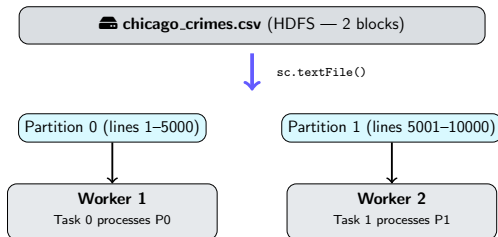
numSlices = Partitions

Controls parallelism. Default from HDFS = number of HDFS blocks.

Each Line = One Element

textFile reads line-by-line. Each line becomes one string element in the RDD.

RDD Partitions — Visual



What Students Should Understand:

- 1 HDFS block $\approx$ 1 partition (by default)
- 1 partition $\rightarrow$ 1 task
- Tasks run in parallel across workers
- No machine loads the entire dataset

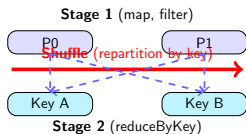
Important Notes:

- Partitions control parallelism
- More partitions $\neq$ always faster
- Shuffle operations may create new partitions
- Poor partition sizing $\rightarrow$ performance issues

Rule of Thumb

Aim for **2–4 partitions per CPU core**. Balance parallelism and scheduling overhead.

Understanding Shuffle in Spark



Triggers: reduceByKey, groupByKey, join, distinct, repartition

Impact:

- Creates a **new stage** boundary
- Involves **network transfer** between executors
- May **spill to disk** if data exceeds memory

What is Shuffle?

- Redistribution of data across partitions
- Groups records with the **same key** together
- Happens in **wide transformations**

Spark vs MapReduce Shuffle

MapReduce	Spark
Every job: Map → Shuffle → Reduce	Only when needed (DAG-based)
Always disk-based	Can stay in memory
Fixed pipeline	Optimized by Catalyst

Tip: Prefer reduceByKey over groupByKey — it combines locally **before** shuffling.

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions**
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary

Transformations — Building the DAG (Lazy)

A transformation takes an RDD and returns a **new** RDD. **No computation happens yet!**

Transformation	Description	Example
<code>map(f)</code>	Apply <code>f</code> to every element	Parse CSV $\rightarrow$ tuple
<code>filter(f)</code>	Keep elements where <code>f</code> is True	Remove header row
<code>flatMap(f)</code>	Map then flatten (1-to-many)	Sentences $\rightarrow$ words
<code>distinct()</code>	Deduplicate elements	Unique crime types
<code>union(other)</code>	Merge two RDDs	Combine two files

Why Lazy?

Spark optimizes the full pipeline *before* executing. Unnecessary computations are skipped entirely.

Key-Value Transformations

When your RDD contains (key, value) tuples, additional transformations unlock:

Transformation	Description	Example
<code>reduceByKey(f)</code>	Combine values per key	Sum counts per district
<code>groupByKey()</code>	Group all values per key	All crimes per district
<code>sortByKey()</code>	Sort RDD by key	Alphabetical types
<code>join(other)</code>	Inner join on key	Crimes $\bowtie$ Districts
<code>mapValues(f)</code>	Apply <code>f</code> to values only	Keep keys unchanged

Shuffle Warning

`reduceByKey`, `groupByKey`, `join`, and `sortByKey` all trigger a **shuffle** — data is exchanged across the network. This is the most expensive operation in Spark.

Actions — Triggering Execution (Eager)

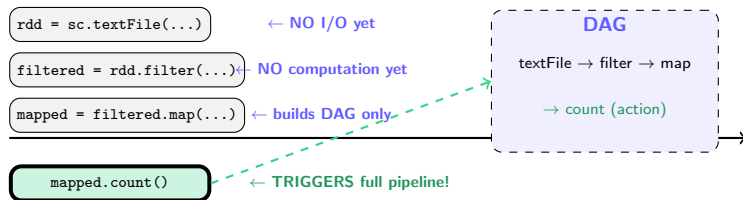
An action **triggers computation** of the entire DAG and returns a result to the Driver.

Action	Returns	Use Case
<code>count()</code>	Integer	How many records?
<code>first()</code>	Element	Inspect first record
<code>take(n)</code>	List of <code>n</code>	Preview first <code>n</code> records
<code>collect()</code>	List (all)	Bring all data to Driver
<code>reduce(f)</code>	Single value	Sum / max over all elements
<code>saveAsTextFile(p)</code>	None	Write RDD to HDFS / local
<code>countByKey()</code>	Dict	Count per key

Never Do This On Big Data

`collect()` sends **all** data to the Driver. On a 50 GB dataset this will crash your Driver with `OutOfMemoryError`. Use `take(n)` instead for previews.

Lazy Evaluation — Why It Matters



Benefits of lazy evaluation:

- Spark optimizes the *full pipeline* before executing anything
- If `filter` eliminates 95% of data, `map` never processes those rows
- Transformations are **pipelined** within tasks — no intermediate materialization

RDD Lineage — toDebugString()

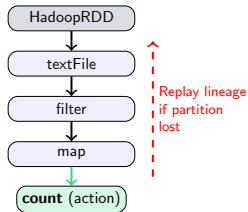
Every RDD knows its **parent** — the chain of transformations that created it.

```
>>> mapped.toDebugString()  
(2) PythonRDD[5] at count  
  | MapPartitionsRDD[4] at map  
  | PythonRDD[3] at filter  
  | hdfs:///data/crimes.csv  
  | HadoopRDD[0]
```

Reading bottom-up:

- 1 Load from HDFS (HadoopRDD)
- 2 Parse CSV (textFile)
- 3 Apply filter transformation
- 4 Apply map transformation
- 5 Trigger with count (action)

Lineage Graph (DAG):



Fault Tolerance Without Replication

If a partition of map RDD is lost, Spark **replays the lineage**: re-read from HDFS → filter → map. No data replication needed — the lineage *is* the recovery plan.

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale**
- 6 Spark Web UI
- 7 Summary

PairRDD: The Foundation of Aggregation

A **Pair RDD** is an RDD of (key, value) tuples — the foundation for all group-by analytics.

```

rdd = sc.textFile("hdfs:///crimes.csv")
header = rdd.first()

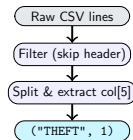
# Create Pair RDD: (Primary Type, 1)
pairs = (rdd
    .filter(lambda x: x != header)
    .map(lambda x: (x.split(",")[5], 1))
)
# ("THEFT", 1), ("BATTERY", 1), ...

```

Why Pair RDDs?

- Spark has **no tables or schemas** at the RDD level
- To group, aggregate, or join, Spark needs a **key** to partition data by
- Pair RDDs give every record an explicit key — enabling `reduceByKey`, `groupByKey`, `join`

What happens inside:



Pattern: Create Pairs → Aggregate by Key → Sort Results

PairRDD in Action: Crime Type Counts

Goal: Count crimes by type and find the top 5 most frequent.

```
# Step 1: Create Pair RDD
pairs = (rdd
    .filter(lambda x: x != header)
    .map(lambda x: (x.split(",")[5], 1))
)

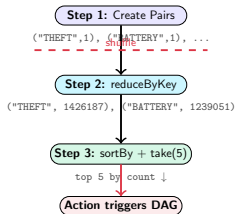
# Step 2: Aggregate by key
counts = pairs.reduceByKey(lambda a, b: a + b)

# Step 3: Sort descending, get top 5
top5 = (counts
    .sortBy(lambda x: x[1], ascending=False)
    .take(5)
)
```

Output:

```
[('THEFT',          1_426_187),
 ('BATTERY',        1_239_051),
 ('CRIMINAL DAMAGE', 771_743),
 ('NARCOTICS',       714_921),
 ('ASSAULT',         437_815)]
```

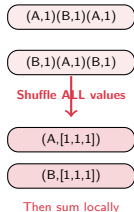
Data flow — step by step:



Key insight: reduceByKey combines locally on each partition first (like a combiner), then shuffles — much less data than groupByKey.

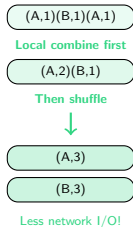
reduceByKey vs groupByKey — Critical Difference

groupByKey()



- Shuffles **ALL** values across network
- Memory-heavy: materializes all values
- Risk of OutOfMemoryError

reduceByKey()



- Applies **combiner locally** first
- Shuffles only partial aggregates
- Much more memory-efficient

Rule

Always prefer reduceByKey for associative + commutative operations (sum, count, max).

Chicago Crimes — Full RDD Pipeline

```

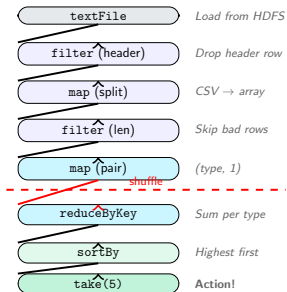
rdd = sc.textFile("hdfs:///crimes.csv")
header = rdd.first()

crime_counts = (
    rdd
    .filter(lambda x: x != header)
    .map(lambda x: x.split(","))
    .filter(lambda f: len(f) > 5)
    .map(lambda f: (f[5], 1))
    .reduceByKey(lambda a, b: a+b)
    .sortBy(lambda x: x[1],
            ascending=False)
)

crime_counts.take(5)

```

Reading the pipeline:



Under the Hood: Two Stages

Stage 0 (no shuffle): textFile → filter → split → filter → pair + local combine counts → sort → take(5) returns to Driver

Stage 1 (after shuffle): merge

Chicago Crimes — Full RDD Pipeline Explained

```

rdd = sc.textFile("hdfs:///crimes.csv")
header = rdd.first()

crime_counts = (
    rdd
    .filter(lambda x: x != header)
    .map(lambda x: x.split(","))
    .filter(lambda f: len(f) > 5)
    .map(lambda f: (f[5], 1))
    .reduceByKey(lambda a, b: a+b)
    .sortBy(lambda x: x[1],
            ascending=False)
)
crime_counts.take(5)
```

What is lambda?

A **lambda** is a one-line anonymous function:

`lambda input: output`

Same as writing `def f(input): return output`
but without giving it a name.

Decoding each lambda:

1. `lambda x: x != header`
`x` = one line of text
Returns True/False → keeps or drops
2. `lambda x: x.split(",")`
`x` = "THEFT,2024,..."
Returns ["THEFT", "2024", ...]
3. `lambda f: len(f) > 5`
`f` = the array from step 2
Drops rows with ≤ 5 fields
4. `lambda f: (f[5], 1)`
Extracts column 5 (crime type)
Returns ("THEFT", 1)
5. `lambda a, b: a + b`
`a, b` = two counts for same key
Returns their sum
6. `lambda x: x[1]`
`x` = ("THEFT", 2124)
Sorts by index 1 (the count)

Pipeline summary: 

Example 2: Arrests per District

```
# Which districts have the highest arrest rates?
rdd = sc.textFile("hdfs:///data/chicago_crimes.csv")
header = rdd.first()

arrest_by_district = (
    rdd
    .filter(lambda x: x != header)
    .map(lambda x: x.split(","))
    .filter(lambda f: len(f) > 8)
    .map(lambda f: (f[4], int(f[7] == "true"))) # (District, 0/1)
    .aggregateByKey(
        (0, 0), # (total, arrests)
        lambda acc, v: (acc[0]+1, acc[1]+v),
        lambda a, b: (a[0]+b[0], a[1]+b[1])
    )
    .mapValues(lambda v: round(v[1]/v[0]*100, 1))
    .sortBy(lambda x: x[1], ascending=False)
)

arrest_by_district.take(5)
# [('18', 34.2), ('1', 31.7), ('12', 28.5), ...]
```

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI**
- 7 Summary

Reading the Spark Web UI

<http://master:4040>

Jobs Stages Storage Environment Executors

Job 0: count at <stdin>:1
Status: SUCCEEDED
Stages: 2 (Stage 0: 2 tasks, Stage 1: 1 task)
Duration: 3.2 s
Shuffle Write: 1.2 KB Shuffle Read: 1.2 KB

Key Metrics to Watch

- **Stages:** how many stages? (= how many shuffles + 1)
- **Tasks per stage:** = number of partitions at that point
- **Shuffle Write/Read:** data sent across the network
- **Duration:** spot slow (straggler) tasks

Data Locality in the Web UI

Rank	Locality Level	Meaning
☆☆☆	PROCESS_LOCAL	Data in same Executor JVM (cached)
☆☆	NODE_LOCAL	Data on same physical node
☆	RACK_LOCAL	Data on same rack
!	ANY	Data on a different rack (slowest)

Why it matters:

- ANY tasks are slow — data moves across the network
- Spark waits briefly for a better locality slot before downgrading
- Config: `spark.locality.wait`

Lab Question

What % of your tasks were `NODE_LOCAL`?
Check the **Stages** tab → task detail.

Outline

- 1 Recap & Motivation
- 2 Spark Cluster Architecture
- 3 RDDs: The Core Abstraction
- 4 Transformations & Actions
- 5 Key-Value RDDs: Aggregation at Scale
- 6 Spark Web UI
- 7 Summary**

Key Takeaways

- 1 **Spark** replaces MapReduce's disk-based model with **in-memory** processing
- 2 **Architecture:** Driver → Cluster Manager → Workers with Executors
- 3 **RDDs:** immutable, lazy, distributed, fault-tolerant via lineage
- 4 **Transformations** build a DAG (lazy); **Actions** trigger execution (eager)
- 5 `reduceByKey` $\gg$ `groupByKey` for aggregation
- 6 **Web UI:** read jobs, stages, tasks, shuffle metrics at port 4040



Remember

MapReduce	Read → Process → Write to Disk → Repeat
Spark	Read → Transform in RAM → Transform → Action

Next Session: Spark DataFrames & Spark SQL

Week 7B: Higher-Level APIs — From RDDs to DataFrames

- DataFrames: distributed tables with named, typed columns
- `spark.read.csv()`, `json()`, `parquet()` loaders
- Transformations: `select`, `filter`, `groupBy`, `agg`, `join`
- **Spark SQL**: register views and query with standard SQL
- **Catalyst optimizer**: why DataFrames are often faster than RDDs

Homework / Lab Prep

- SSH to cluster: `pyspark --master yarn`
- Run: `rdd = sc.textFile("hdfs:///data/chicago_crimes.csv"); rdd.count()`
- Expected: **10001** (header + 10000 data rows)

Lambda Expressions — The Basics

A **lambda** is a small, anonymous (unnamed) function written in one line.

Syntax:

```
lambda input: output
```

Equivalent named function:

```
def my_func(input):  
    return output
```

Key Rule

A lambda can only have **one expression**.

No loops, no if/else blocks, no multi-line logic.

Examples — building intuition:

Double a number:

```
lambda x: x * 2
```

Input: 5 → Output: 10

Check if even:

```
lambda x: x % 2 == 0
```

Input: 7 → Output: False

Get string length:

```
lambda s: len(s)
```

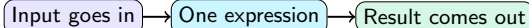
Input: "hello" → Output: 5

Two inputs (add):

```
lambda a, b: a + b
```

Input: 3, 7 → Output: 10

Think of it as:



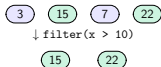
Lambda in Spark — Why and How

Spark passes your lambda to **every element** in the RDD. The lambda tells each worker *what to do* with each record.

filter — keep or drop?

```
# Lambda returns True/False
# True  -> keep the element
# False -> drop it

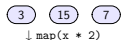
rdd.filter(lambda x: x > 10)
```



map — transform each element

```
# Lambda transforms every element
# Input element -> Output element

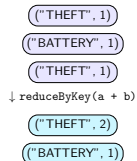
rdd.map(lambda x: x * 2)
```



reduceByKey — combine by key

```
# Lambda takes TWO values
# with the same key and
# combines them into one

rdd.reduceByKey(lambda a, b: a + b)
```



Mental Model

filter → "Should I keep this?" (Yes/No)

map → "Transform this into that"

reduceByKey → "Merge values with same key"